Polygon Clipping

Here is a description of a recursive Sutherland-Hodgman polygon clipping algorithm that works with homogeneous 3D attributed coordinates.

Boundary type

There needs to exist a mapping between the integers and the clipping boundaries of the unit cube. These boundaries are referred to as the left, right, top, bottom, front, and back boundaries. The order is unimportant, but in the discussion that follows, it will be assumed that a sequence does exist. The discussion will refer to the first and last boundaries of the sequence. It may be helpful to create an enumerated data type containing symbolic constants for the various boundaries.

poly_clip()

This is the main function for clipping a polygon. It takes an array of attributed points as the as the polygon to be clipped (and most likely an integer indicating the number of points in the polygon), and another attributed point array that will contain the clipped polygon vertices when the function is done. The function should return an integer indicating the final number of vertices in the clipped polygon.

```
Variables:   an array of attr_points for the first point
             seen by each clipping pipeline stage

             an array of flags to indicate whether the
             the pipeline stage has been seen, i.e., whether
             the first point is valid or uninitialized garbage

             an array of attr_point for the last point
             seen by each clipping pipeline stage

       The size of each of these arrays is the number of stages in
       the clipping pipeline, one stage for each clipping boundary.

       The output array, the integer count, the first, flag, and
       last arrays will be referred to collectively as "extra
       stuff "

       These arrays should be available to many of the functions
       described here.  They can either be passed explicitly as
       function arguments or defined as global variables to this
       module.


The flag array should be initialized.

Loop over all the vertex points in pIn
     clip a point(pIn[k], FirstBoundary, extra stuff);

clip the last point(extra stuff);

return the count
```

clip a point

This function does most of the work. It takes primarily as input a point p, and a boundary b to clip against. It works with the edge defined by p and the last point seen at this boundary stage of the clip.

```
    if this is the first time a point has been seen at stage b
        first[b] = p;
        Update the flag array as well

    else
        // Previous point exists

        if the edge defined by p and last[b] crosses boundary b
             ipt = intersect(p, last[b], b);
             send ipt to the next stage of the pipeline or to the
                 output array depending on b, i.e.
                 clip a point (iPt, b+1, extra stuff) // (1)
                   or
                 pOut[count++] = iPt;

    // Save the most recent vertex seen at this stage.
    last[b] = p;

    if p is inside boundary b then
         send p to the next stage or to the
             output array, like above  // (2)
```

clip last point

This function is very similar to the clip a single point routine, but instead of working with the edge defined by the point passed in and the last point seen, it works with the edge defined by the first point seen and the last point seen.

```
     Loop over the boundaries with b
         if first[b] exists and the edge defined by first[b] and
             last[b] crosses boundary b then

             ipt = intersect(last[b], first[b], b);

             send ipt to the next stage of the pipeline or to the
                 output array  // (3)
```

intersect

Takes two attributed points and a boundary and returns an attributed point. The intersection point can be found using the approach taken for line clipping: Calculate alpha using the appropriate boundary coordinates for the points and boundary in question and interpolate between the points. Depending on the boundary, the formula for alpha does not need all of the boundary coordinates, so don't compute them all. Use only what's needed for the given boundary. When interpolating, interpolate all of the coordinates of the points.

inside

Takes an attributed point and a boundary. By using the selected boundary, this function computes the boundary coordinate of the point for that boundary and returns whether or not the point is inside the boundary.

cross

Takes two attributed points and a boundary. Simply calculates the "inside" values above for the two points and returns true or false depending on whether the values are equal or not.